Table of Contents

I. LEARNING OBJECTIVES 3
II. INTRODUCTION.....3
III. TARGET LABORATORY SOURCE CODE.....3
IV. LABORATORY PROCEDURES AND TASKS 4
Task A: Building with Debugging Symbols.....4
Task B: Setting Boundaries and Runtime Positioning 4
Task C: Mapping Static BSS Segment Bounds.....5
readelf -l: to read progam header.....5
Compare the results with the above:.....6
Task D: Extracting OS Kernel Address Tables 7
Task E: Auditing Local Register States and Low-Level Cells.....8
POST-LAB TECHNICAL ASSESSMENT QUESTIONS.....9

Table of Figures

Figure 1: Target Mapping 3
Figure 2: Compiling 4
Figure 3: gdb Session 4
Figure 4: Configuring Breakpoint 4
Figure 5: Query 5
Figure 6: readelf -l 5
Figure 7: Matching the address 6
Figure 8: Virtual Mapping Configuration 7
Figure 9: Local Stack Workspace 8
Figure 10: Examining Heap data pool 8
Figure 11: Quit 8

Mapping Process Runtime Topography Using Low-Level Hardware Debugging Infrastructure

I. LEARNING OBJECTIVES

By completing this laboratory assignment, students will be able to:

1) Evaluate the physical address boundaries separating static allocations (.bss) from dynamic regions (Heap, Stack).

2) Navigate runtime program contexts using specialized subcommands inside the GNU Debugger (GDB).

II. INTRODUCTION

Operating systems virtualize physical RAM to afford each running application an isolated, structured sandbox known as a virtual process address space. Within this layout, variable state information is rigorously segmented. Unit variables whose lifetimes span the entire lifecycle of the software settle into fixed-size boundaries like the BSS segment. Conversely, deep operational variables rely on dynamic runtimes like the hardware managed Stack or the programmer policed Heap. To fully understand low-level compute environments, software engineers must look past high-level programming language code constructs. This exercise uses hardware debugging techniques to extract memory tables, map storage regions, and examine literal hex payloads embedded directly inside system RAM.

III. TARGET LABORATORY SOURCE CODE

You must replicate the architectural code implementation listed in Listing 1. Save this implementation locally on your Linux machine as demo.c.



Figure 1: Target Mapping

IV. LABORATORY PROCEDURES AND TASKS

Task A: Building with Debugging Symbols

Compile the source code inside your terminal application using the -g optimization preservation flag. This forces the compiler to embed symbol definitions, variable tracking names, and source file line bindings directly into the compiled executable target container:

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ sudo nano demo.c
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gcc -g demo.c -o demo
```

Figure 2: Compiling

Task B: Setting Boundaries and Runtime Positioning

Initialize the target binary within the debugger environment and position execution anchors to freeze state pools immediately prior to function termination:

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ sudo nano demo.c
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gcc -g demo.c -o demo
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ sudo nano demo.c
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gdb ./demo
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo...
(gdb)
```

Figure 3: gdb Session

Inside the GDB session console, configure a breakpoint at line 23 to suspend state operations while your heap allocations and local registers remain active:

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo...
(gdb) break 23
Breakpoint 1 at 0x1277: file demo.c, line 24.
(gdb) run
Starting program: /home/jibesh-shrestha/Batch38/demo

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n])
```

Figure 4: Configuring Breakpoint

Task C: Mapping Static BSS Segment Bounds

Query GDB to expose the structural mapping arrangement generated during compilation linking phases:

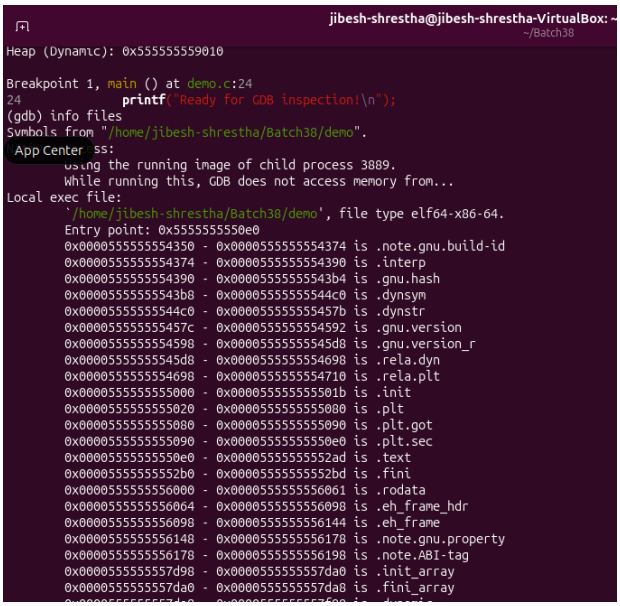


Figure 5: Query

Locate the terminal data row mapping ending explicitly with the identifier .bss. Verify if the absolute pointer position of your global tracking variable resides safely inside this targeted memory range block:

readelf -l: to read progam header

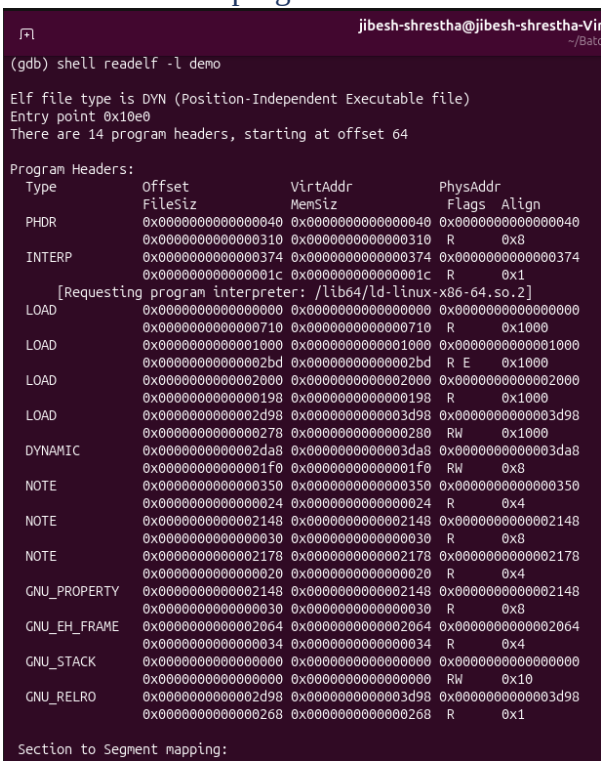


Figure 6: readelf -l

Compare the results with the above:

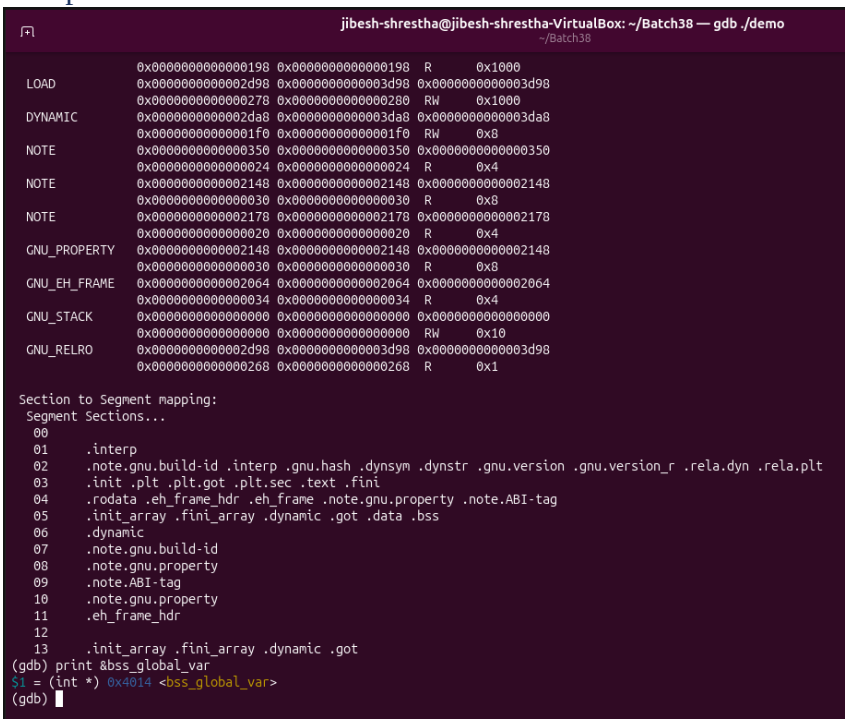


Figure 7: Matching the address

readelf shows the .bss address as 0x4010. This is the address stored in the file, before the program runs.

GDB shows 0x555555558010. This is the real address after the program is loaded into memory.

The difference is 0x555555554000. The system adds this number to every address when it loads the program, because the program is position-independent (PIE).

0x4010 + 0x555555554000 = 0x555555558010

So both show the same place, just before and after loading.

Task D: Extracting OS Kernel Address Tables

Request the live active virtual mapping configuration directly from the operating system kernel:

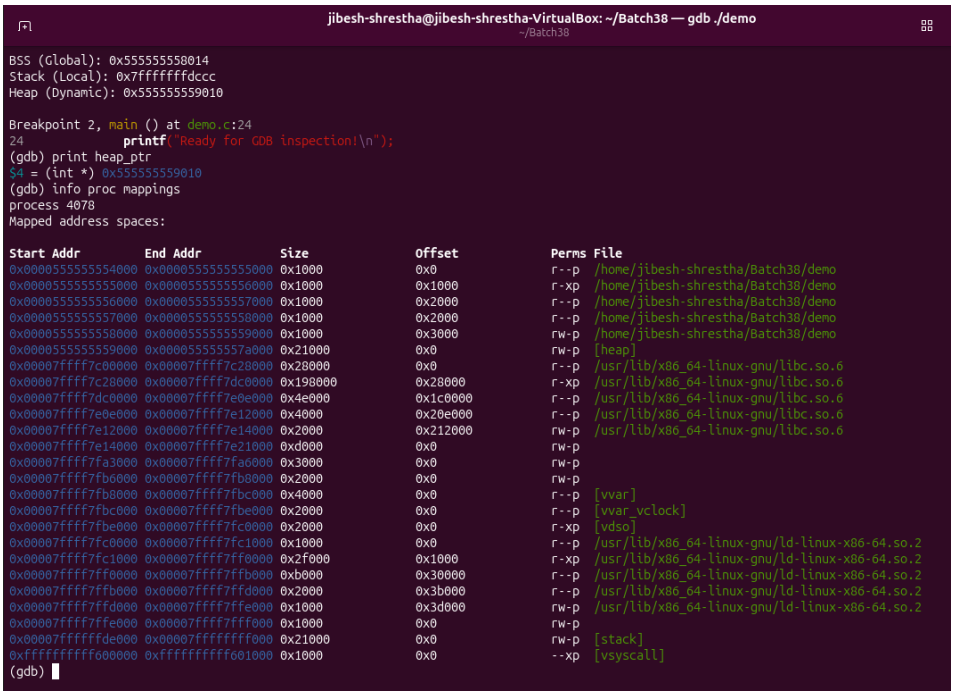


Figure 8: Virtual Mapping Configuration

Isolate the dedicated mapping rows assigned explicitly to the descriptors labeled [heap] and [stack]. Record the upper and lower memory location hex limits for both sections.

- [heap]: lower limit 0x0000555555559000, upper limit 0x000055555557a000
- [stack]: lower limit 0x00007fffffde000, upper limit 0x00007fffffff000
- BSS address 0x555555558014 is inside the demo rw-p mapping (0x555555558000 to 0x555555559000).
- Address Matrix:

Region	Start	End
.bss	0x555555558010	0x555555558014
[heap]	0x0000555555559000	0x000055555557a000
[stack]	0x00007fffffde000	0x00007fffffff000

Task E: Auditing Local Register States and Low-Level Cells

Examine the active variables mapped inside the local stack workspace and extract raw values directly from the CPU's hardware Stack Pointer register ($sp):

```
(gdb) print &stack_local_var
$6 = (int *) 0x7fffffffdccc
(gdb) print stack_local_var
$7 = 42
(gdb) print &stack_local_var
$8 = (int *) 0x7fffffffdccc
(gdb) x/4wx $sp
0x7fffffffdcc0: 0x00000000      0x00000000      0x00000000      0x0000002a
(gdb)
```

Figure 9: Local Stack Workspace

Transition your query to examine the heap data pool. Evaluate the literal dynamic storage state by referencing your tracking handle address:

```
0x7fffffffdcc0: 0x00000000      0x00000000      0x00000000      0x0000002a
(gdb) print heap_ptr
$9 = (int *) 0x555555559010
(gdb) x/1wd heap_ptr
0x555555559010: 99
(gdb)
```

Figure 10: Examining Heap data pool

- x examines memory.
- 1 shows 1 unit.
- w makes the unit a word (4 bytes).
- d prints it in decimal.

So x/1wd heap_ptr reads one 4-byte value at heap_ptr and shows 99

Once completed, let the container terminate cleanly and detach your tracking engines:

```
(gdb) continue
Continuing.
Ready for GDB inspection!
[Inferior 1 (process 4078) exited normally]
(gdb) quit
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

Figure 11: Quit

POST-LAB TECHNICAL ASSESSMENT QUESTIONS

1) Based on the addresses extracted during your lab runtime session, prove mathematically that the stack segment and the heap segment migrate toward one another during process expansion cycles.

From info proc mappings:

- Heap end: 0x55555557a000
- Stack start: 0x7fffffde000

Gap = 0x7fffffde000 − 0x55555557a000 = 0x2AAAAAA64000 (about 42.7 TiB)

- The heap grows up, so its end increases by h.
- The stack grows down, so its start decreases by s.

New gap = old gap − (h + s)

Since h > 0 and s > 0, the new gap is always smaller than the old gap. So the heap and stack move toward each other.

2) If you run this compiled application simultaneously in two separate terminal windows, will both running programs display the exact same address locations or unique addresses? Explain your answer by discussing the concepts of Virtual Memory Spaces and operating system process isolation

Usually different addresses.

- The OS uses ASLR, which randomly changes the addresses each time a program runs.
- Each process has its own private virtual memory.
- Addresses like 0x5555... are virtual, not real RAM addresses.
- The OS maps each process's virtual addresses to different physical RAM.
- Even if two programs show the same address, they use different physical memory.
- One process cannot read or change another process's memory. This is process isolation.